

Name: MD. Abu Taher,

ID: 2102003,

Reg: 10130

Course Code: CIT-322,

Project Title: MiniEduOS (Custom Educational Operating System)



Abstract

MiniEduOS is a small but meaningful educational operating system project built with the purpose of understanding how a real computer system works internally. Instead of only reading theoretical concepts from books, this project focuses on practical implementation starting from the boot process and going all the way to a working minimal user interface. The system demonstrates how a computer transitions from firmware control to a bootloader, and then finally hands over execution to a custom kernel. Inside the kernel, basic system initialization is performed, including screen setup and text rendering using a simple bitmap font system. Everything runs inside a virtual machine environment (QEMU), which makes it safe, repeatable, and easy to debug. Through this project, the hidden layers of an operating system become much more understandable and visual.

Introduction

Operating systems are often treated as complex and abstract systems when studied theoretically. However, when implemented in a simplified form, the internal working becomes surprisingly logical and structured. MiniEduOS was created to bridge this gap between theory and practice. The main idea behind this project is to understand what really happens when a computer starts and how control flows step by step until a usable system appears on the screen. Instead of focusing on advanced production-level features like multitasking or memory protection, this project focuses on core fundamentals such as bootloading, kernel execution, and basic display output. This helps build a strong foundation for deeper operating system concepts in the future.

System Working Flow

The execution of MiniEduOS follows a very structured sequence. When the system is powered on, the firmware performs initial hardware checks and then transfers control to the bootloader (GRUB). The bootloader is responsible for locating the kernel and loading it into memory using the Multiboot specification. Once the kernel is loaded, execution is transferred to it. From this moment, the kernel becomes the main controller of the system. It initializes essential components such as screen output and prepares a simple environment for displaying information. After initialization, the kernel begins rendering text on the screen using a custom bitmap font system. This is the first visible output of the operating system, which confirms that the boot process was successful.

Kernel and Font Rendering System

The kernel in MiniEduOS is intentionally minimal, but it plays a very important role. Its main responsibility is to initialize the system and handle output to the display. Instead of using external graphics libraries, the project uses a custom 8x8 bitmap font system. Each character is represented using 8 bytes, where each byte corresponds to a row of pixels. This allows direct manipulation of screen output at a very low level. Although simple, this method closely

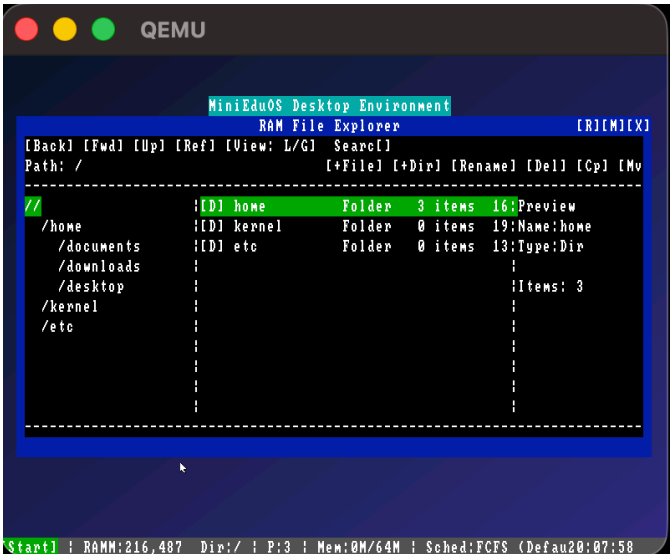
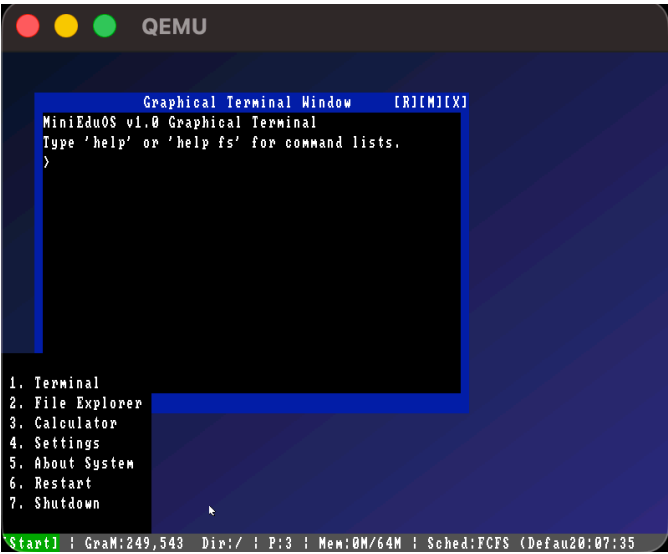
reflects how early operating systems handled text rendering. It also helps in understanding how characters are actually drawn on the screen rather than just displayed magically.

Testing Environment

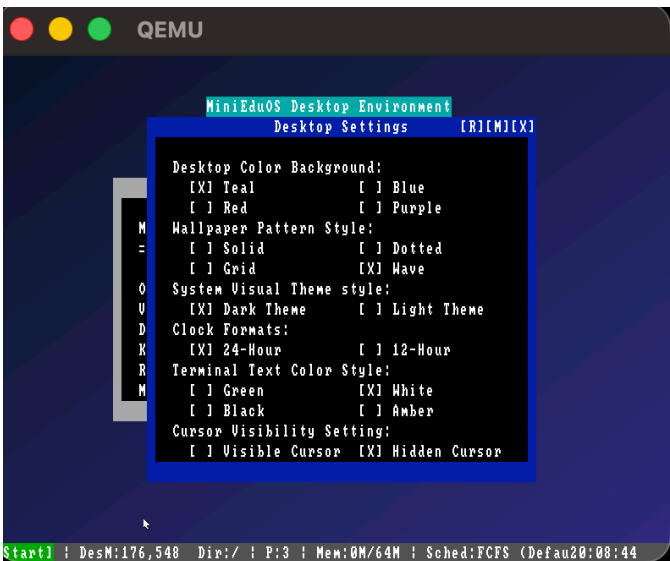
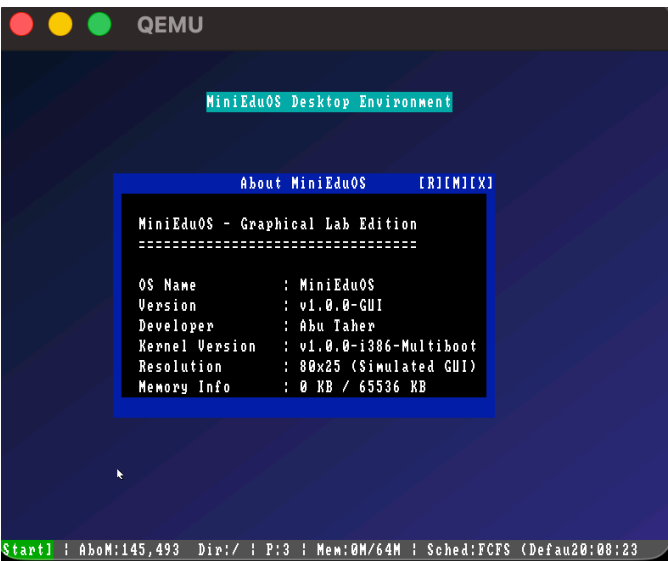
The entire system is executed inside the QEMU virtual machine. This ensures that the operating system can be tested safely without affecting real hardware. To make testing easier and more efficient, a Python automation script is used. This script is responsible for launching the OS, capturing screenshots, and simulating keyboard inputs. This automated workflow helps in quickly verifying changes and ensures that every component of the system behaves as expected.

User Interface (UI)

The UI of MiniEduOS is simple but represents different functional modules of a real operating system. Each screen reflects a specific part of system interaction.



The first image shows the Terminal environment along with a Start button interface. This part represents the main interaction point of the system where basic commands and navigation features are available. It gives the feeling of a minimal operating environment where the user can interact with the system directly. The second image shows the File Explorer module. This component allows users to browse system files and understand how file navigation works inside an operating system. Even though it is simplified, it represents the core idea of file management.



The third image shows the About MiniEduOS section. This part contains system-related information such as project identity and version details. It acts as a small informational window that describes the system itself. The fourth image shows the Settings module. This section allows configuration of system-level options. Although minimal, it demonstrates how operating systems typically provide control over system behavior.

Project Timeline (Gantt Chart)

The project was completed within a very short time frame of one week. Each day was dedicated to a specific part of development, ensuring steady progress and clear milestones.

Task / Day	Day 1	Day 2	Day 3	Day 4	Day 5	Day 6	Day 7
Planning & System Design	✓						
Bootloader Configuration (GRUB)		✓					
Kernel Development			✓				
Basic Display & Font System				✓			
UI Module Development					✓		
Testing with QEMU + Debugging						✓	
Final Optimization & Report Writing							✓

Discussion

Working on MiniEduOS provided a very practical understanding of how operating systems function internally. Concepts that often feel difficult in theory—such as boot sequences, kernel initialization, and hardware interaction became much clearer through implementation. One of the most interesting parts of the project was seeing how a simple kernel can take control of the system after the bootloader finishes its job. Even though the system is minimal, it still follows the same structural principles used in modern operating systems. This project also improved understanding of low-level programming, system design thinking, and debugging techniques in a virtual environment.

Conclusion

MiniEduOS successfully demonstrates the basic but essential workflow of an operating system. It shows how a system transitions from bootloader to kernel and finally to a working interface. While it is not a full-scale operating system, it provides a strong foundation for learning advanced topics such as process scheduling, memory management, and device drivers in the future. Overall, this project serves as a practical step toward understanding real-world operating system design.

References

[1] Intel Multiboot Specification [2] GNU GRUB Documentation [3] OSDev Wiki (<https://wiki.osdev.org>) [4] QEMU Virtual Machine Documentation